

MAE 106 Final Report

Lab 3A Group 1

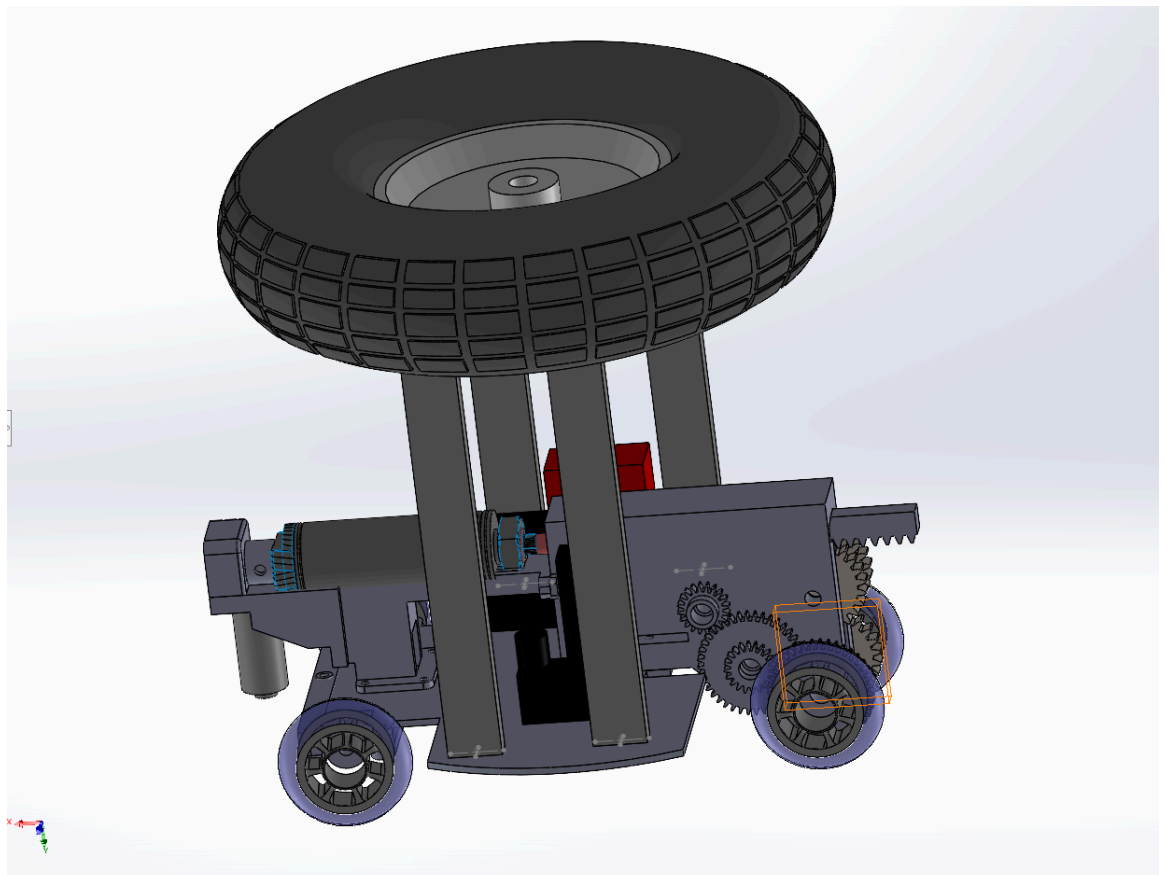
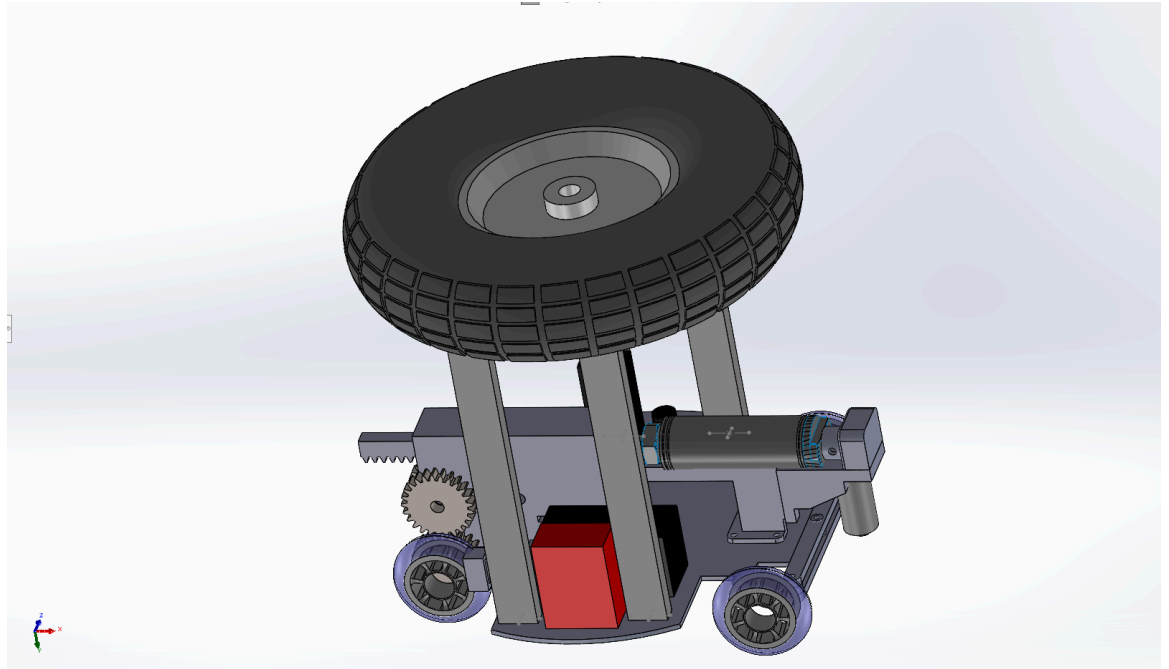
Adam Diersing, Logan Bigler, and Martin Li

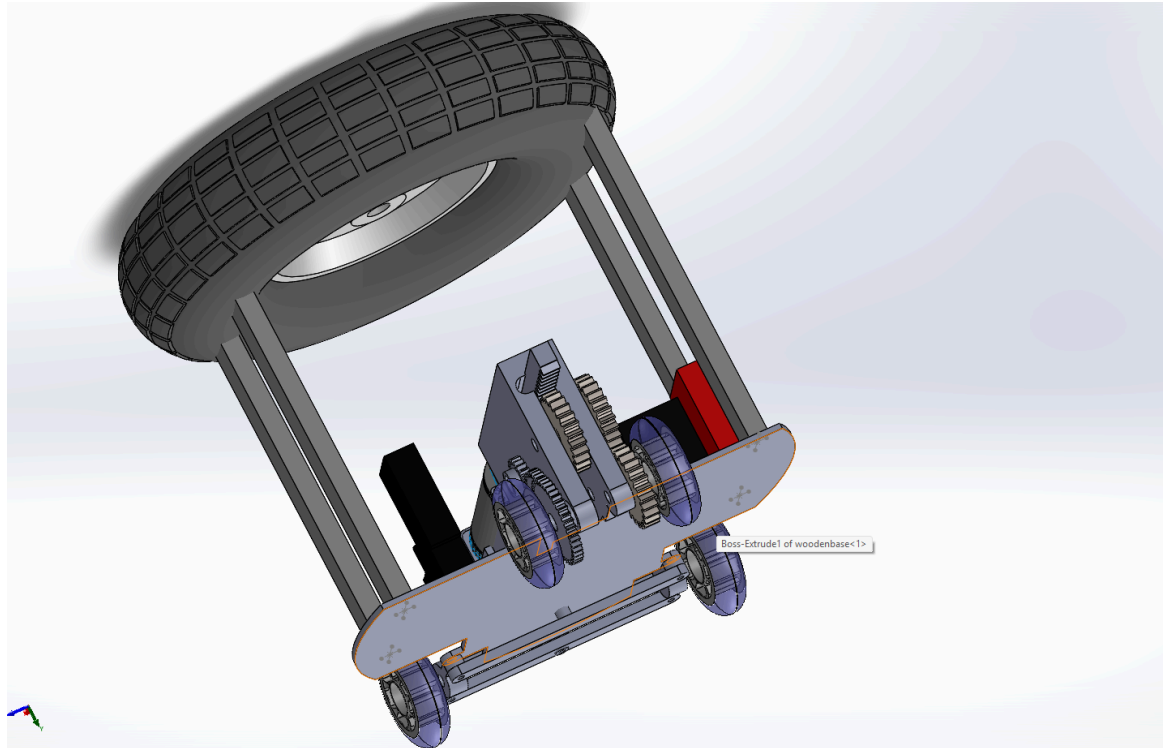
A. System Description

1. The main goal of our mechanical design was to take advantage of the small amount of energy the spring produces when it's pulling the piston back, and putting that into the forward motion of the car.
2. Our robot was different since we had 2 separate gear trains being driven by a rack and pinion gear system, one side getting energy from the piston pushing forward, and the other side getting energy from the piston pulling back. Since the piston firing is where most of the energy is coming from, we put a 3:1 gear ratio on that side which favored speed over torque, and the other side was a 1:4 gear ratio to favor the torque over speed since the only thing pulling the piston back in was the spring.

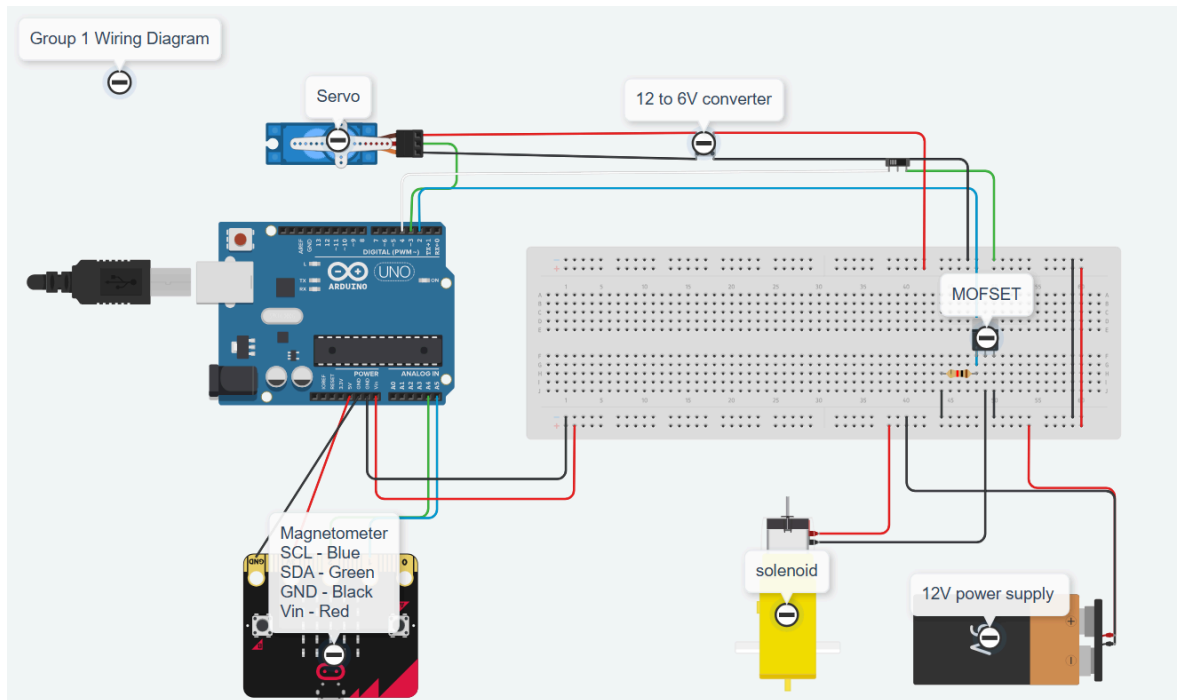


- 3.
- 4.

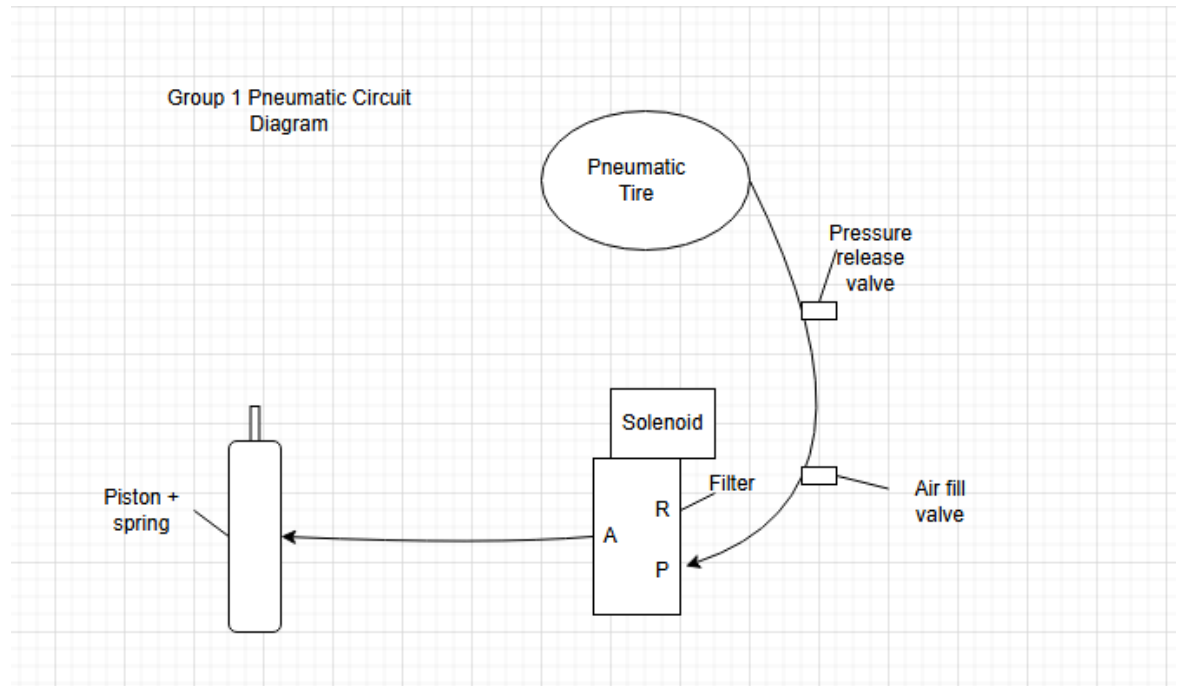




5.



6.



7. Will add final code at the end of report.
- 8.

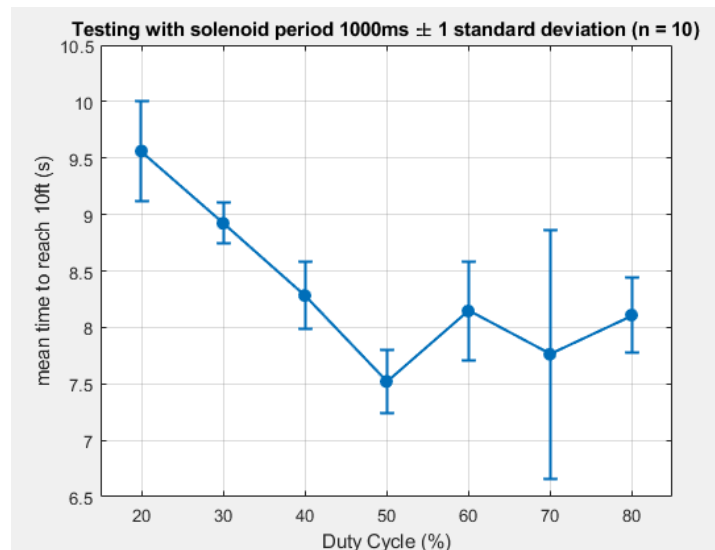
Item	Quantity	Unit Cost	Total
MOFSET	1	\$1.00	\$1.00
Aluminum Rod 8x300mm	3	\$3.25	\$9.75
One-Way Bearing	7	\$1.40	\$9.80
Subtotal			\$20.55

Did all 3D printing at home

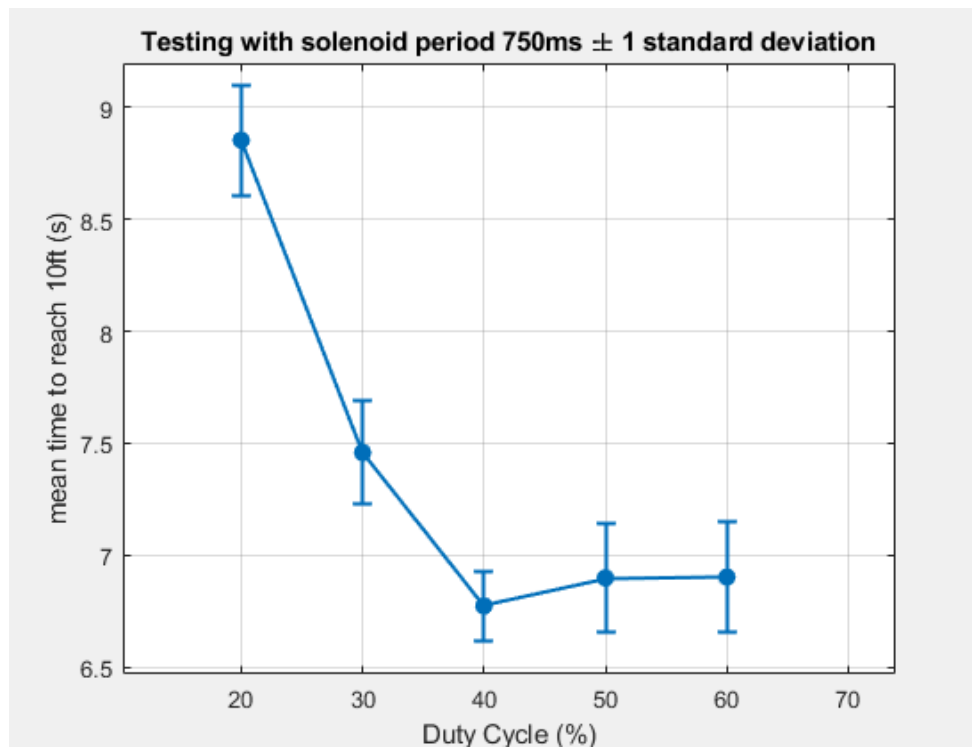
B. Testing and Modeling

1. We chose to optimize the speed of the robot by adjusting performance of the pneumatic cylinder. We defined speed by the time it takes for the robot to travel 10ft.
2. We chose to vary the duty cycle of the pneumatic cylinder. We kept the starting tire pressure at 40Psi for testing, the Kp at 0.6 for heading corrections, and kept solenoid period constant at 1000ms for a range of duty cycles. Then tested at a period of 750ms for a range of duty cycles, all other parameters constant.
- 3.

Duty cycle with 1s period												
%	Run 1	Run 2	Run 3	Run 4	Run 5	Run 6	Run 7	Run 8	Run 9	Run 10	Mean	std dev
20	9.25	9.45	9.36	9.1	10.66	9.57	9.43	9.29	9.86	9.65	9.562	0.4419351385
30	8.68	9.15	9.06	8.73	8.89	8.75	9.11	8.92	9.14	8.79	8.922	0.1817690843
40	8.16	8.83	8.3	8.06	7.91	8.34	8.47	8.13	7.98	8.65	8.283	0.2965748922
50	7.63	7.68	7.7	7.63	6.78	7.56	7.37	7.73	7.59	7.54	7.521	0.2795412909
60	8.83	8.75	7.66	7.7	7.68	8.36	8.36	8.21	8.15	7.75	8.145	0.4393745555
70	10.55	8.8	6.95	7.45	7.36	7.46	7.36	7.23	7.34	7.12	7.762	1.100240378
80	8.73	8.43	7.7	7.8	7.99	8.14	8.17	8.03	7.75	8.35	8.109	0.3271917549



0.75s period												
%	Run 1	Run 2	Run 3	Run 4	Run 5	Run 6	Run 7	Run 8	Run 9	Run 10	Mean	std dev
20	8.46	8.85	9.2	8.95	9.01	8.67	8.54	9.12	8.75	8.99	8.854	0.2451 84737 6
30	7.31	7.21	7.9	7.47	7.64	7.15	7.34	7.37	7.58	7.62	7.459	0.2284 46444
40	6.98	6.81	6.55	6.77	6.68	6.89	6.82	6.95	6.51	6.78	6.774	0.1560 05697 9
50	7.13	6.6	6.9	6.93	6.87	7.13	6.95	7.27	6.69	6.52	6.899	0.2416 35630 2
60	6.76	6.83	6.88	6.38	7.05	6.78	7.31	7.12	6.97	6.95	6.903	0.2485 53593 6

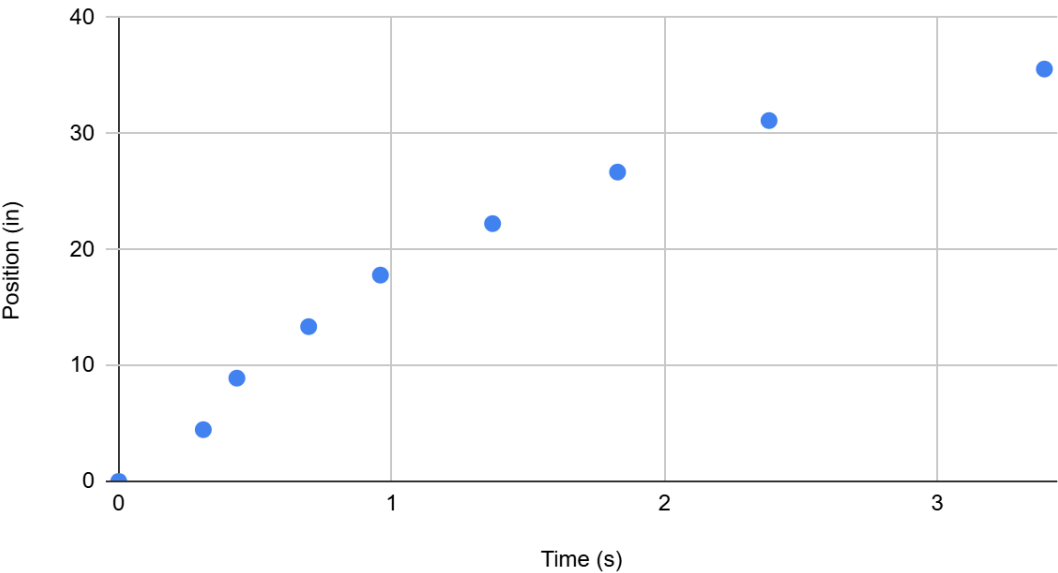


4. We observed that using a 40% duty cycle for the solenoid with a period of 750ms was the fastest among the tested values, and also had less variation in the

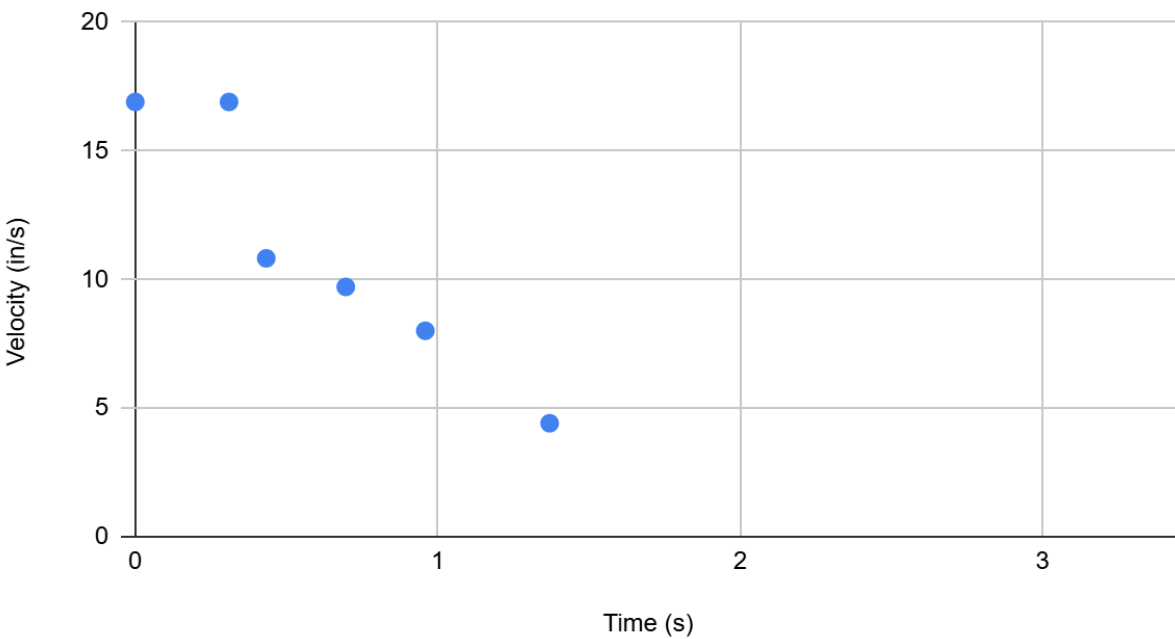
runs, with the least standard deviation through 10 runs (0.156). We also observed more stable movement at this duty cycle.

5.

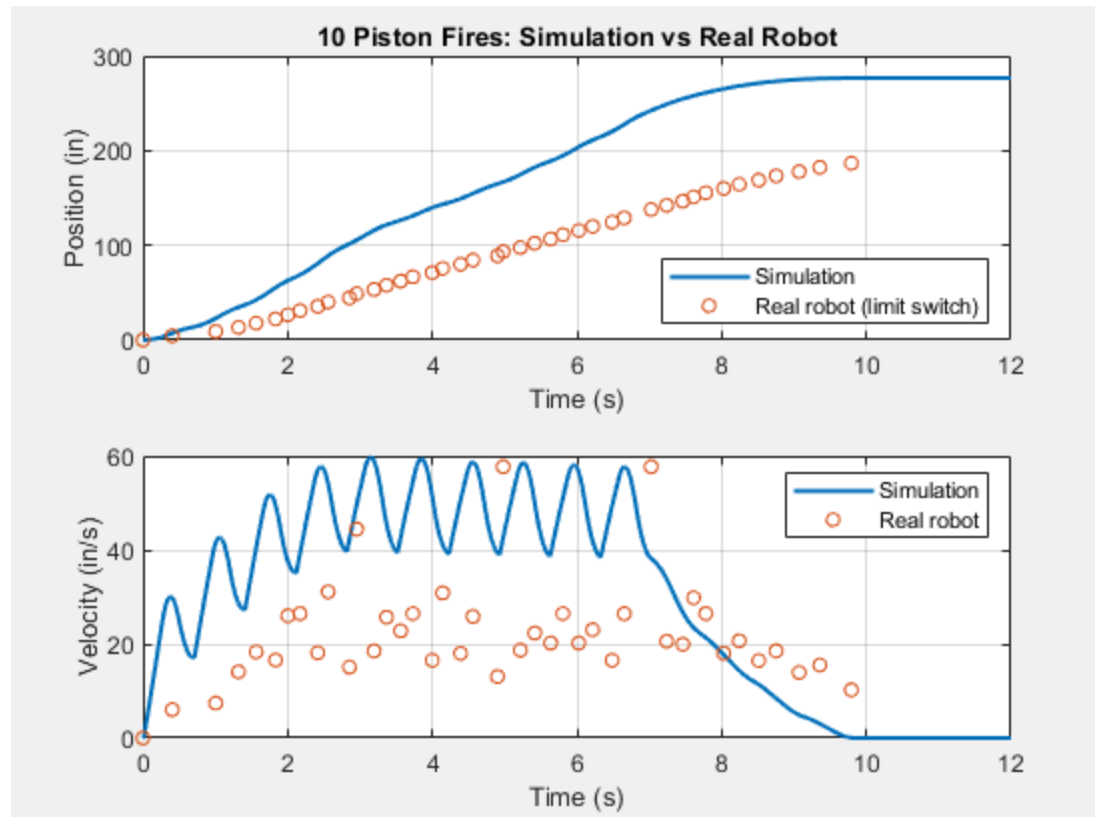
Position vs Time for 1 impulse



Velocity vs Time for 1 impulse



6.



7. The figure above compares the simulator's prediction for 10 piston fires against the robot's actual measured response to 10 piston fires. The simulator was calibrated using the experimental single impulse response from the robot. The simulation predicted a final displacement of 266in while the robot actually traveled 186.7in. The pressure in the tire decreases with every actuation, but likely decreases more in reality than what the simulation accounts for. There are more resistive losses at higher speeds, so the faster moving robot sheds more energy than simulated.

C. Teamwork

1. Contributions:

- **Adam:** I did all of the soldering and wiring for the robot as the electrical role and made sure the electronics were functioning throughout testing of the robot. I noticed at one point that the MOFSET was not working and replaced it. I helped with assembly of the robot - cutting wood for pillars and putting components on the board. Also helped with reassembly after replacing the aluminum rod support columns with wood. I designed and printed the brackets for electrical parts as well as printing parts for the second version of our rack and pinion mechanism. I fixed the final code to ensure that the robot ran properly for the demo and took the video for the demo. I did the necessary experimental

testing and took data for the report, and used it to optimize the robot's speed for the demo. I also did all of section B and some of section A for this report.

- **Martin:** I participated in our team project by cutting and sanding wood for our team. I helped cut the top and bottom circular plates and the support columns for the robot. I also coded the main program for our robot. It took some calibration and adjustments to get the numbers right, but with Adam's help, everything worked out. Shoutout to Adam. Another time, the support columns made with 3D-printed parts and aluminum rods broke, so I replaced them with a pair of wooden columns.
- **Logan:** Since my job was the mechanical side of things, I designed and tweaked parts in CAD until everything meshed together well to make the robot run. I also worked on the gear ratios on both sides of the gear train to help favor torque or speed when needed. Outside of mechanical, I helped with the initial base code we made including the calibration of the servo, and I helped find the ideal times for the piston to fire.

2. Strategy:

Strategy1:

I know this is probably repeated every quarter, but it is important to start early. Even with these repeated reminders, many students still make the same mistakes. Instead of only telling students to start early, I would recommend moving the deadline to an earlier date.

The first few weeks of the quarter are often relatively relaxed, and students usually have more free time. After that, because some classes have one midterm while others have two, students often have midterms almost every week, making the schedule much busier.

If the physical robot work were moved to the first few weeks of the quarter and the theoretical content were taught afterward, I think students would be more motivated and able to work in a less stressful environment.

Strategy2:

I think the second most important strategy in teamwork is communication. It is important for group members to communicate information with each other. Some important information that should be shared includes time availability, progress during work, and difficulties encountered during work. Having this information communicated thoroughly will help the team collaborate and keep their eyes "open."

When information is not communicated within a group, one could feel blind and have no idea what is going on. In a broader sense, information is very important in any environment, and helping students develop an instinct to seek and communicate information will be very helpful not only in this robot project but also in their futures.

Our strategies involved testing parts out constantly, since sometimes it works in CAD while it doesn't work in real life. For example, the first rack I made for the rack and pinion gear

relationship didn't work since it was connected to an overhang and the piston fired with a larger force than it could take. My other strategy was to start as early as I possibly could, since there was a lot of work that had to be done for the method of propulsion I came up with to be successful. Also, starting early gives your other group members time to complete their part as well, since some pieces can't be fully tested without a semi-functioning robot.

FINAL CODE:

```
#include <Servo.h>
#include <Wire.h>
#include <LIS3MDL.h>
#include <LSM6.h>

// =====
// MAE 106 FINAL PROJECT RACE CODE
// Lab 3A Group 1
// =====
//
// Final race behavior:
// 1. 15s delay after turning on
// 2. LEG 1: drives straight on its starting heading for 10 feet
//    (measured by the limit switch on the wheel).
// 3. TURN 1: RIGHT 90 degrees.
// 4. LEG 2: drives straight on the new heading for 40 feet.
// 5. TURN 2: LEFT 90 degrees.
// 6. LEG 3: drives straight on the final heading for 10 feet.
// 7. Robot stops (course complete), or earlier at the 58 s timer.
//
//
// =====

// =====
// OBJECTS
// =====

Servo steeringServo;
LIS3MDL mag;
LSM6 imu;
```

```

// =====
// PIN SETTINGS
// =====

const int SERVO_PIN = 3;
const int SOLENOID_PIN = 2;
const int REED_PIN = 4;
const int LED_PIN = 13;


// =====
// SOLENOID SETTINGS
// =====

// If solenoid stops working, switch this true/false.
const bool SOLENOID_ACTIVE_HIGH = true;

// solenoid duty cycle of 40% with period of 750ms, most optimal for speed
// from testing
const unsigned long SOLENOID_ON_TIME = 300;
const unsigned long SOLENOID_OFF_TIME = 450;

bool solenoidState = LOW;
unsigned long lastSolenoidTime = 0;


// =====
// SERVO / STEERING SETTINGS
// =====

// Adjust SERVO_CENTER until wheels are physically straight.
const int SERVO_CENTER = 90;

// Narrow range for final race so it does not lock hard right/left.
const int SERVO_MIN = 70;
const int SERVO_MAX = 110;

// SERVO SLEW-RATE LIMIT: maximum change in servo angle per control
// loop (~20 ms). 4 deg/loop = ~200 deg/s. This turns sudden steering
// commands (especially the snap when the turn ends) into smooth

```

```

// ramps so the chassis does not get rocked.
// Lower it if the robot still lurches; raise it if steering feels
// too sluggish to hold heading.
const int SERVO_SLEW_DEG_PER_LOOP = 4;

// Ignore heading errors smaller than this so the servo is not
// constantly twitching around zero.
const float ERROR_DEADBAND_DEG = 2.0;

// Where the servo actually is right now (slew limiting state).
int currentServoAngle = SERVO_CENTER;

// All steering goes through this: moves the real servo toward the
// requested angle no faster than the slew limit.
void writeServoSlewed(int targetAngle) {
    targetAngle = constrain(targetAngle, 0, 180);
    int delta = targetAngle - currentServoAngle;
    if (delta > SERVO_SLEW_DEG_PER_LOOP) delta = SERVO_SLEW_DEG_PER_LOOP;
    if (delta < -SERVO_SLEW_DEG_PER_LOOP) delta = -SERVO_SLEW_DEG_PER_LOOP;
    currentServoAngle += delta;
    steeringServo.write(currentServoAngle);
}

const bool REVERSE_STEERING = false;

// straight-line gain.
float Kp = 0.6;

// Maximum correction from center while driving straight.
// This prevents the robot from staying hard right/left.
const float MAX_CORRECTION = 20.0;

// Stronger steering used ONLY while executing the 90 degree turn.
const float TURN_Kp = 1.2;
const float TURN_MAX_CORRECTION = 40.0;
const int TURN_SERVO_MIN = 50;    // wider throw allowed during turn
const int TURN_SERVO_MAX = 130;
const int TURN_MAX_ARC_IN = 100;

```

```

// =====
// TARGET HEADING SETTINGS
// =====

// For final race, use starting direction as target.
// Aim the robot down the trench before pressing reset.
const bool USE_STARTING_HEADING_AS_TARGET = true;

// Only used if USE_STARTING_HEADING_AS_TARGET is false.
float TARGET_HEADING_DEG = 0.0;

// =====
// NAVIGATION / COURSE SETTINGS
// =====
//
// THE COURSE, defined as a list of (straight leg, then turn):
//   leg 1: 10 ft straight, then turn RIGHT 90
//   leg 2: 40 ft straight, then turn LEFT 90, go 10ft and stop

const int NUM_TURNS = 2;

const float LEG_DISTANCE_IN[NUM_TURNS] = { 120.0, 480.0 };
const float TURN_ANGLES_DEG[NUM_TURNS] = { 90.0, -90.0 };

const float FINAL_LEG_DISTANCE_IN = 125.0;
const float TURN_COMPLETE_TOLERANCE_DEG = 15;
bool turnErrorWasPositive = true;

int turnIndex = 0;
int drivePhase = 0;
float legStartDistance = 0.0; // odometer reading when this leg began
float turnStartDistance = 0.0; // odometer reading when this turn began

// =====
// RACE TIMING SETTINGS
// =====

```

```

// Press/reset Arduino slightly AFTER "Clear!".
// Robot starts 15 seconds later, at "Go!".
const unsigned long START_DELAY_MS = 15000;

// Stop before the 60 second "Stop!" signal.
const unsigned long RUN_TIME_MS = 58000;

unsigned long programStartTime = 0;
unsigned long robotStartTime = 0;

bool robotStarted = false;
bool robotStopped = false;
bool servoAttached = false;

// =====
// DISTANCE / REED SWITCH SETTINGS
// =====

const float WHEEL_DIAMETER_IN = 2.83;

// The limit/reed switch CHANGES STATE every HALF revolution
// (off -> on at one half, on -> off at the next). Counting every
// state change therefore gives 2 pulses per full revolution.
const int PULSES_PER_REV = 2;

const float DISTANCE_PER_PULSE_IN = 3.14159 * WHEEL_DIAMETER_IN /
PULSES_PER_REV;

long reedPulseCount = 0;
float distanceInches = 0.0;

int lastReedState = HIGH;
unsigned long lastReedTime = 0;
const unsigned long REED_DEBOUNCE_MS = 50;

// =====
// SENSOR STATUS

```

```

// =====

bool magOK = false;
bool imuOK = false;

// =====
// MAGNETOMETER CALIBRATION
// =====

LIS3MDL::vector<int16_t> m_min = { -4172, -2362, +3681 };
LIS3MDL::vector<int16_t> m_max = { -567, +1034, +4187 };

// =====
// FUNCTION DECLARATIONS
// =====

void turnSolenoidOn();
void turnSolenoidOff();

void startRobot();
void stopRobot();

void updateSolenoid();
void updateSteering();
void updateReedSwitch();
void updateNavigation();
void printStatus();

float computeHeading();
template <typename T> float computeHeadingFrom(LIS3MDL::vector<T> from);
float wrapAngle180(float angle);

// =====
// SETUP
// =====

void setup() {

```

```

Serial.begin(9600);
Wire.begin();

pinMode(SOLENOID_PIN, OUTPUT);
pinMode(REED_PIN, INPUT_PULLUP);
pinMode(LED_PIN, OUTPUT);

turnSolenoidOff();

Serial.println("MAE 106 Final Race Code Starting...");
Serial.println("Waiting 15 seconds before actuation.");

magOK = mag.init();
if (magOK) {
    mag.enableDefault();
    Serial.println("LIS3MDL magnetometer detected.");
} else {
    Serial.println("WARNING: LIS3MDL magnetometer not detected.");
}

imuOK = imu.init();
if (imuOK) {
    imu.enableDefault();
    Serial.println("LSM6 IMU detected.");
} else {
    Serial.println("WARNING: LSM6 IMU not detected.");
}

programStartTime = millis();

Serial.println("Setup complete. Press/reset should happen after Clear
signal.");
}

// =====
// MAIN LOOP
// =====

```

```

void loop() {
    unsigned long currentTime = millis();

    if (robotStopped) {
        turnSolenoidOff();

        if (servoAttached) {
            steeringServo.write(SERVO_CENTER);
        }

        return;
    }

    // Wait 15 seconds after reset before actuating.
    if (!robotStarted) {
        if (currentTime - programStartTime >= START_DELAY_MS) {
            startRobot();
        } else {
            // During countdown: no solenoid, no servo movement.
            turnSolenoidOff();

            static unsigned long lastCountdownPrint = 0;
            if (currentTime - lastCountdownPrint >= 1000) {
                Serial.print("Countdown seconds left: ");
                Serial.println((START_DELAY_MS - (currentTime - programStartTime))
/ 1000);
                lastCountdownPrint = currentTime;
            }

            return;
        }
    }

    updateReedSwitch();
    updateNavigation();
    updateSteering();
    updateSolenoid();
    printStatus();

    // Stop before the official 60 second stop signal.

```

```

    if (currentTime - robotStartTime >= RUN_TIME_MS) {
        stopRobot();
    }

    delay(20);
}

// =====
// START ROBOT
// =====

void startRobot() {
    robotStarted = true;
    robotStartTime = millis();
    lastSolenoidTime = millis();

    // Attach servo only when race starts.
    steeringServo.attach(SERVO_PIN);
    servoAttached = true;
    steeringServo.write(SERVO_CENTER);

    delay(100);

    if (magOK && imuOK) {
        mag.read();
        imu.read();

        if (USE_STARTING_HEADING_AS_TARGET) {
            TARGET_HEADING_DEG = computeHeading();
        }
    }

    Serial.println("ROBOT STARTED.");
    Serial.print("Target heading: ");
    Serial.println(TARGET_HEADING_DEG);
}

// =====

```

```

// STOP ROBOT
// =====

void stopRobot() {
    robotStopped = true;

    turnSolenoidOff();

    if (servoAttached) {
        steeringServo.write(SERVO_CENTER);
    }

    Serial.println("ROBOT STOPPED.");
    Serial.print("Final distance: ");
    Serial.print(distanceInches);
    Serial.println(" inches");

    Serial.print("Final reed pulses: ");
    Serial.println(reedPulseCount);
}

// =====
// SOLENOID CONTROL
// =====

void turnSolenoidOn() {
    if (SOLENOID_ACTIVE_HIGH) {
        digitalWrite(SOLENOID_PIN, HIGH);
    } else {
        digitalWrite(SOLENOID_PIN, LOW);
    }

    digitalWrite(LED_PIN, HIGH);
}

void turnSolenoidOff() {
    if (SOLENOID_ACTIVE_HIGH) {
        digitalWrite(SOLENOID_PIN, LOW);
    }
}

```

```

    } else {
        digitalWrite(SOLENOID_PIN, HIGH);
    }

    digitalWrite(LED_PIN, LOW);
}

void updateSolenoid() {
    unsigned long currentTime = millis();

    if (solenoidState == LOW) {
        if (currentTime - lastSolenoidTime >= SOLENOID_OFF_TIME) {
            solenoidState = HIGH;
            turnSolenoidOn();
            lastSolenoidTime = currentTime;
        }
    } else {
        if (currentTime - lastSolenoidTime >= SOLENOID_ON_TIME) {
            solenoidState = LOW;
            turnSolenoidOff();
            lastSolenoidTime = currentTime;
        }
    }
}

void finishTurn(const char *reason) {
    drivePhase = 0;
    turnIndex++;
    legStartDistance = distanceInches;

    Serial.print(">>> TURN ");
    Serial.print(turnIndex);           // just finished (1-based)
    Serial.print(" COMPLETE ");
    Serial.print(reason);
    Serial.println(".");
    Serial.print(">>> Driving leg ");
    Serial.println(turnIndex + 1);
}

```

```

void updateNavigation() {
    // All turns done: drive the final leg, then stop.
    if (turnIndex >= NUM_TURNS) {
        if (distanceInches - legStartDistance >= FINAL_LEG_DISTANCE_IN) {
            Serial.println(">>> FINAL LEG DONE. COURSE COMPLETE.");
            stopRobot();
        }
        return;
    }

    // ---- Driving a straight leg: trigger the next turn ----
    if (drivePhase == 0) {
        if (distanceInches - legStartDistance >= LEG_DISTANCE_IN[turnIndex]) {
            drivePhase = 1;
            turnStartDistance = distanceInches;

            // Rotate the target heading by this turn's angle, kept 0-360.
            float newTarget = TARGET_HEADING_DEG + TURN_ANGLES_DEG[turnIndex];
            while (newTarget >= 360.0) newTarget -= 360.0;
            while (newTarget < 0.0) newTarget += 360.0;
            TARGET_HEADING_DEG = newTarget;

            turnErrorWasPositive = (TURN_ANGLES_DEG[turnIndex] > 0);

            Serial.print(">>> LEG ");
            Serial.print(turnIndex + 1);
            Serial.print(" DONE. STARTING TURN ");
            Serial.print(turnIndex + 1);
            Serial.print(TURN_ANGLES_DEG[turnIndex] > 0 ? " (RIGHT " : " (LEFT
");
            Serial.print(abs(TURN_ANGLES_DEG[turnIndex]));
            Serial.println(" deg).");
            Serial.print(">>> New target heading: ");
            Serial.println(TARGET_HEADING_DEG);
        }
        return;
    }

    // ---- Turning: decide when the turn is complete ----

```

```

if (drivePhase == 1) {
    // Safety bail-out: never circle forever.
    if (distanceInches - turnStartDistance >= TURN_MAX_ARC_IN) {
        finishTurn("BAIL-OUT max arc");
        return;
    }

    if (magOK && imuOK) {
        float error = wrapAngle180(TARGET_HEADING_DEG - computeHeading());

        // Done if the heading is within tolerance of the target
        bool withinTolerance = (abs(error) <= TURN_COMPLETE_TOLERANCE_DEG);

        // or if the error has crossed through zero
        bool crossedTarget =
            (turnErrorWasPositive ? (error < 0.0) : (error > 0.0))
            && (abs(error) < 90.0);

        if (withinTolerance || crossedTarget) {

            finishTurn(crossedTarget && !withinTolerance ? "crossed target"
                : "within
tolerance");
        }
    }
}

}

// =====
// STEERING FEEDBACK CONTROL
// =====

void updateSteering() {
    if (!servoAttached) {
        return;
    }
}

```

```

bool turning = (drivePhase == 1);

if (!(magOK && imuOK)) {
    if (turning) {
        // No compass: steer hard over in this turn's direction.
        // Heading increases clockwise, so a positive angle = right.
        // The servo is upside down, so REVERSE_STEERING flips the sign.
        int idx = constrain(turnIndex, 0, NUM_TURNS - 1);
        float hardOver = (TURN_ANGLES_DEG[idx] > 0) ? TURN_MAX_CORRECTION
                                                    : -TURN_MAX_CORRECTION;

        if (REVERSE_STEERING) {
            hardOver = -hardOver;
        }
        int servoAngle = constrain((int)(SERVO_CENTER + hardOver),
                                   TURN_SERVO_MIN, TURN_SERVO_MAX);

        writeServoSlewed(servoAngle);
    } else {
        writeServoSlewed(SERVO_CENTER);
    }
    return;
}

mag.read();
imu.read();

float heading = computeHeading();

// Error from target heading.
float error = wrapAngle180(TARGET_HEADING_DEG - heading);

// Deadband: treat tiny errors as zero so the servo is not
// constantly hunting. (Not applied during the turn.)
if (!turning && abs(error) < ERROR_DEADBAND_DEG) {
    error = 0.0;
}

// Use stronger gain and wider throw while turning.
float gain      = turning ? TURN_Kp : Kp;
float maxCorr   = turning ? TURN_MAX_CORRECTION : MAX_CORRECTION;
int servoMin    = turning ? TURN_SERVO_MIN : SERVO_MIN;

```

```

int servoMax  = turning ? TURN_SERVO_MAX : SERVO_MAX;

// Proportional control.
float correction = gain * error;

// Limit correction so it does not stay full right or full left.
correction = constrain(correction, -maxCorr, maxCorr);

// Flip because the servo is mounted upside down.
if (REVERSE_STEERING) {
    correction = -correction;
}

int servoAngle = SERVO_CENTER + correction;
servoAngle = constrain(servoAngle, servoMin, servoMax);

writeServoSlewed(servoAngle);
}

// =====
// REED SWITCH DISTANCE MEASUREMENT
// =====

void updateReedSwitch() {
    int reedState = digitalRead(REED_PIN);
    unsigned long currentTime = millis();

    // The switch CHANGES STATE every half revolution, so count
    // EVERY state change (both LOW->HIGH and HIGH->LOW edges).
    // Each counted change = half a wheel revolution.
    if (reedState != lastReedState) {
        if (currentTime - lastReedTime > REED_DEBOUNCE_MS) {
            reedPulseCount++;
            distanceInches = reedPulseCount * DISTANCE_PER_PULSE_IN;
            lastReedTime = currentTime;
            lastReedState = reedState;
        }
    } else {
        lastReedState = reedState;
    }
}

```

```

    }
}

// =====
// SERIAL MONITOR OUTPUT - for debugging
// =====

void printStatus() {
    static unsigned long lastPrintTime = 0;
    unsigned long currentTime = millis();

    if (currentTime - lastPrintTime >= 500) {
        float heading = -1;
        float error = 0;

        if (magOK && imuOK) {
            heading = computeHeading();
            error = wrapAngle180(TARGET_HEADING_DEG - heading);
        }

        Serial.print("Time: ");
        Serial.print((currentTime - robotStartTime) / 1000.0);

        Serial.print(" s | Heading: ");
        Serial.print(heading);

        Serial.print(" deg | Target: ");
        Serial.print(TARGET_HEADING_DEG);

        Serial.print(" deg | Error: ");
        Serial.print(error);

        Serial.print(" deg | Distance: ");
        Serial.print(distanceInches);

        Serial.print(" in | Pulses: ");
        Serial.print(reedPulseCount);

        Serial.print(" | Phase: ");

```

```

    if (drivePhase == 1) {
        Serial.print("TURN");
        Serial.print(turnIndex + 1);
    } else {
        Serial.print("LEG");
        Serial.print(turnIndex + 1);
    }

    Serial.print(" | Solenoid: ");
    Serial.print(solenoidState == HIGH ? "ON" : "OFF");

    Serial.print(" | magOK: ");
    Serial.print(magOK);

    Serial.print(" | imuOK: ");
    Serial.println(imuOK);

    lastPrintTime = currentTime;
}
}

// =====
// ANGLE HELPER
// =====

float wrapAngle180(float angle) {
    while (angle > 180.0) {
        angle -= 360.0;
    }

    while (angle < -180.0) {
        angle += 360.0;
    }

    return angle;
}

// =====

```

```

// HEADING CALCULATION
// =====

float computeHeading() {
    // This assumes the +X direction of the sensor board points forward.
    // If heading behaves strangely, try changing this axis.
    return computeHeadingFrom(LIS3MDL::vector<int> { 1, 0, 0 });
}

template <typename T>
float computeHeadingFrom(LIS3MDL::vector<T> from) {
    LIS3MDL::vector<int32_t> temp_m = { mag.m.x, mag.m.y, mag.m.z };
    LIS3MDL::vector<int16_t> a = { imu.a.x, imu.a.y, imu.a.z };

    temp_m.x -= ((int32_t)m_min.x + m_max.x) / 2;
    temp_m.y -= ((int32_t)m_min.y + m_max.y) / 2;
    temp_m.z -= ((int32_t)m_min.z + m_max.z) / 2;

    LIS3MDL::vector<float> E;
    LIS3MDL::vector<float> N;

    LIS3MDL::vector_cross(&temp_m, &a, &E);
    LIS3MDL::vector_normalize(&E);

    LIS3MDL::vector_cross(&a, &E, &N);
    LIS3MDL::vector_normalize(&N);

    float heading = atan2(
        LIS3MDL::vector_dot(&E, &from),
        LIS3MDL::vector_dot(&N, &from)
    ) * 180.0 / PI;

    if (heading < 0) {
        heading += 360.0;
    }

    return heading;
}

```